

# App Dev Project Report

## 1. Student Details

**Name:** Nitesh Sharma

**Roll Number:** 23f2003665

**Email:** 23f2003665@ds.study.iitm.ac.in

**About Me:** Student at IIT Madras BS Degree program, currently in Diploma Level 6th term.

---

## 2. Project Details

**Project Title:** Placement Portal Application -V2

### **Problem Statement:**

We are required to build a **Placement Portal Application (PPA)** web application that allows **Admin (Institute), Companies, and Students** to interact with the system based on their roles.

### **Approach:**

Built using Flask for backend, Vue.js for frontend. Three roles — Admin, Company, Student. Started with database models, then auth, then dashboards, then Celery jobs, then Redis caching. Used MailHog for local email testing

---

## 3. AI/LLM Declaration

I have used an AI assistant during the development of this project. The extent of AI usage is approximately 40-45%. AI was used for debugging, understanding complex library configurations like Flask-Security and Celery, UI styling, a little bit in core logic too.

---

## 4. Technologies and Frameworks Used

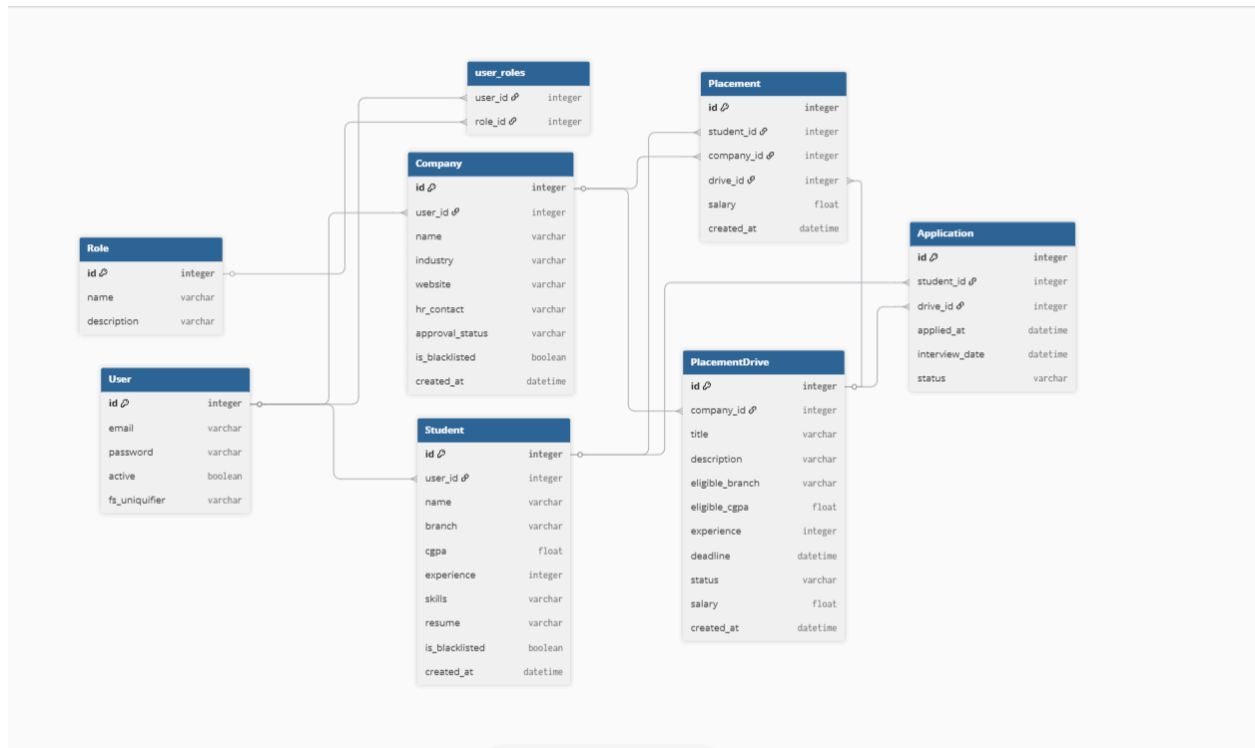
- Flask – Core backend web framework, handles all API routes
  - Flask-Security – User authentication, password hashing, role management
  - SQLAlchemy – ORM for defining database models and running queries
  - SQLite – Lightweight local database for storing all app data
  - Vue.js 3 CDN – Frontend Single Page Application, handles all UI
  - Bootstrap 5 – UI styling, tables, cards, buttons and responsive layout
  - Redis – Caching layer for API responses and message broker for Celery
  - Celery – Asynchronous background job processing
  - Flask-Caching – Redis-backed caching for Flask API endpoints
  - Flask-Mail – Sending emails from Celery tasks
  - MailHog – Local SMTP server for capturing and testing emails
  - Jinja2 – used only for rendering the single HTML entry point
- 

## 5. Database Schema / ER Diagram

**Tables:** User, Role ,user\_roles,Company,Student, PlacementDrive,Application,Placement.

### Relationships:

- User - Company — One to One (one user account per company)
- User - Student — One to One (one user account per student)
- User - Role — Many to Many via user\_roles table
- Company - PlacementDrive — One to Many (one company, many drives)
- Student - Application — One to Many (one student, many applications)
- PlacementDrive - Application — One to Many (one drive, many applicants)
- Student + Company + Drive — Placement this ties all three together when student is selected



## 6. API Resource Endpoints

### Authentication

- **POST /api/register/student** — Register a new student
- **POST /api/register/company** — Register a new company
- **POST /api/login** — Login for all roles
- **GET /api/verify** — Check if session is valid
- **POST /api/logout** — Logout and clear session

### Admin

- **GET /api/admin/dashboard** — Get total counts (Redis cached)
- **GET /api/admin/students** — List all students (Redis cached)
- **GET /api/admin/companies** — List all companies (Redis cached)
- **GET /api/admin/drives** — List all drives (Redis cached)
- **GET /api/admin/applications** — List all applications
- **GET /api/admin/search** — Search students or companies (Redis cached)
- **PUT /api/admin/company/<id>/approve** — Approve or reject a company
- **PUT /api/admin/drive/<id>/approve** — Approve or reject a drive

- **PUT /api/admin/student/<id>/blacklist** — Block or unblock a student
- **PUT /api/admin/company/<id>/blacklist** — Block or unblock a company
- **PUT /api/admin/student/<id>/deactivate** — Activate or deactivate student
- **PUT /api/admin/company/<id>/deactivate** — Activate or deactivate company

### Company

- **GET /api/company/profile/<user\_id>** — Get company profile
- **GET /api/company/drives/<company\_id>** — Get company's drives
- **POST /api/company/drive/create** — Create a new placement drive
- **PUT /api/company/drive/<id>/close** — Close a drive
- **GET /api/company/applications/<company\_id>** — View all received applications
- **PUT /api/company/application/<id>/status** — Update application status
- **GET /api/company/placements/<company\_id>** — View placed students
- **POST /api/company/export/<company\_id>** — Trigger async CSV export

### Student

- **GET /api/student/profile/<user\_id>** — Get student profile
- **PUT /api/student/profile/update/<student\_id>** — Update student profile
- **GET /api/student/drives** — Get all approved drives
- **POST /api/student/apply** — Apply to a drive
- **GET /api/student/applications/<student\_id>** — View own applications
- **GET /api/student/placements/<student\_id>** — View placement history
- **POST /api/student/export/<student\_id>** — Trigger async CSV export

---

## 7. Architecture and Features (optional)

### Architecture Overview:

- **backend/app.py** – Initializes DB, Redis cache, Flask-Security and registers all routes
- **backend/config.py** – App configuration
- **backend/models.py** – All SQLAlchemy models
- **backend/tasks.py** – Celery tasks
- **backend/routes/auth.py** – Auth routes — register, login, logout, session verify
- **backend/routes/admin.py** – Admin routes — dashboard stats, approve, blacklist,etc.
- **backend/routes/company.py** – Company routes — drives, applications, placements,etc
- **backend/routes/student.py** – Student routes — browse drives, apply, profile update,etc
- **backend/templates/index.html** –Single HTML entry point,entire Vue.js SPA is mounted
- **backend/static/js/auth.js** – Vue methods for login and registration
- **backend/static/js/admin.js** – Vue methods for admin dashboard actions
- **backend/static/js/company.js** – Vue methods for company dashboard actions

- **backend/static/js/student.js** – Vue methods for student dashboard actions

### **Implemented Features:**

- Role-based authentication — Admin, Company, Student with separate dashboards
  - Admin pre-created programmatically — no registration allowed
  - Company approval workflow — admin approves before company can post drives
  - Placement drive lifecycle — pending, approved, closed
  - Application status tracking— Applied, Shortlisted, Interview, Selected, Rejected
  - Eligibility validation — CGPA, branch, experience checked before applying
  - Duplicate application prevention
  - Blacklist and Deactivate — two level access control for companies and students
  - Daily deadline reminder emails via Celery Beat
  - Monthly placement report to admin via email
  - CSV export for students and companies via Celery
  - Redis caching with cache invalidation on data change
  - Session management with version tracking
  - Company CSV export
  - Student can view placed students history separately
  - Company can view their placed students separately
- 

## **8. Video Presentation**

**Video Link:**  **Prodict demo.mp4**